



KKU FOODSHARE

Group 6

MEMBER

นางสาวกัญญาวิ ศรีเหรา 673380026-6

นายปวิศร์ แพงมา 673380047-8

นางสาวรสริน เมืองหงษ์ 673380289-4

นางสาวสโรชา เสาทอง 673380296-7

PROBLEM & SOLUTION

ปัญหาที่พบ (Pain Points)

- Food Waste: อาหารส่วนเกินที่ยังดีอยู่ ถูกทิ้งโดยเปล่าประโยชน์
- Information Fragmentation: ข้อมูลกระจายตามโซเชียล ค้นหาและนัดรับยาก
- Lack of Management System: ขาดระบบตรวจสอบสต็อก สถานะการจอง และการยืนยันสิทธิ์



แนวทางแก้ไข

KKU FoodShare Web App: สื่อกลางลงประกาศ
ค้นหาพิกัด จอง และยืนยันรับอาหารด้วย QR /
Pickup Code



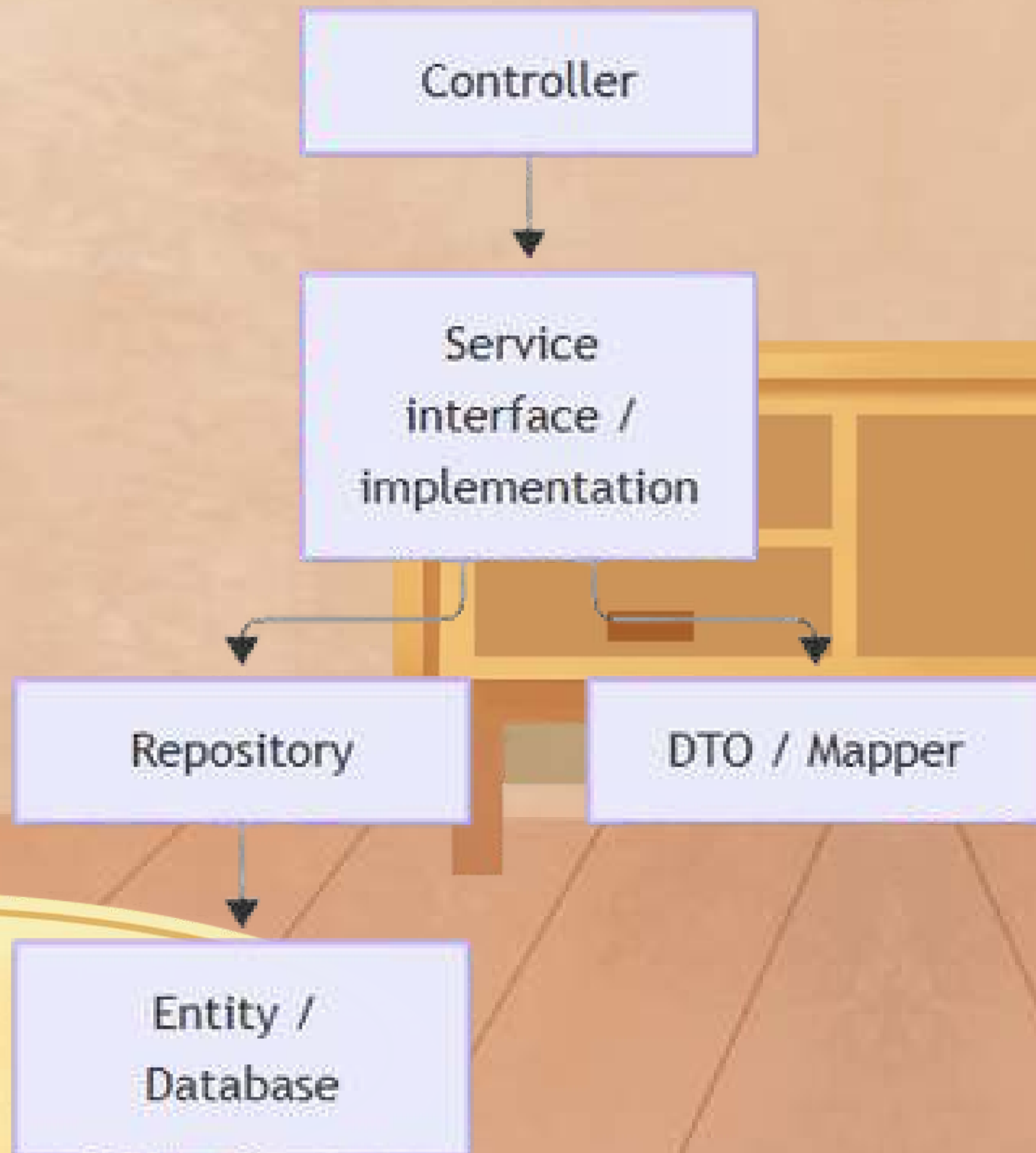
ขอบเขตและฟังก์ชันหลักของระบบ (SYSTEM SCOPE & CAPABILITIES)

กลุ่มผู้ใช้งานเป้าหมาย
นักศึกษา บุคลากร และร้านค้า
ผู้ประกอบการภายใน มข.

4 ฟังก์ชันหลักของระบบ (Core Capabilities)

1. User & Security Management: Authentication (Google OAuth2), Roles (USER/ADMIN), Moderation
2. Food Post & Location: สร้างโพสต์แบ่งปันอาหาร, อัปโหลดรูป, Interactive Map & Route Navigation, Strategy Filter
3. Reservation & Verification: ระบบจองตัดสต็อกอัตโนมัติ, ยืนยันสิทธิ์ด้วย QR / Pickup Code, Notification เตือน
4. Social & Interaction: Comment CRUD (ตอบกลับ), Saved Posts, Report

สถาปัตยกรรมระบบ (LAYERED ARCHITECTURE)



โครงสร้าง 4 ชั้นหลัก (4 Layers)

Presentation Layer

- รับ HTTP Request, ควบคุม Web View & REST API
- @RestController, @Controller

Service Layer

- จัดการ Business Logic, Validation & Transaction
- @Service, @Transactional

Repository Layer

- ติดต่อฐานข้อมูลผ่าน Spring Data JPA
- @Repository, JpaRepository

Domain & DTO Layer

- จัดเก็บ Entities, Enums, DTOs และ Mappers

การประยุกต์ใช้ SOLID PRINCIPLES

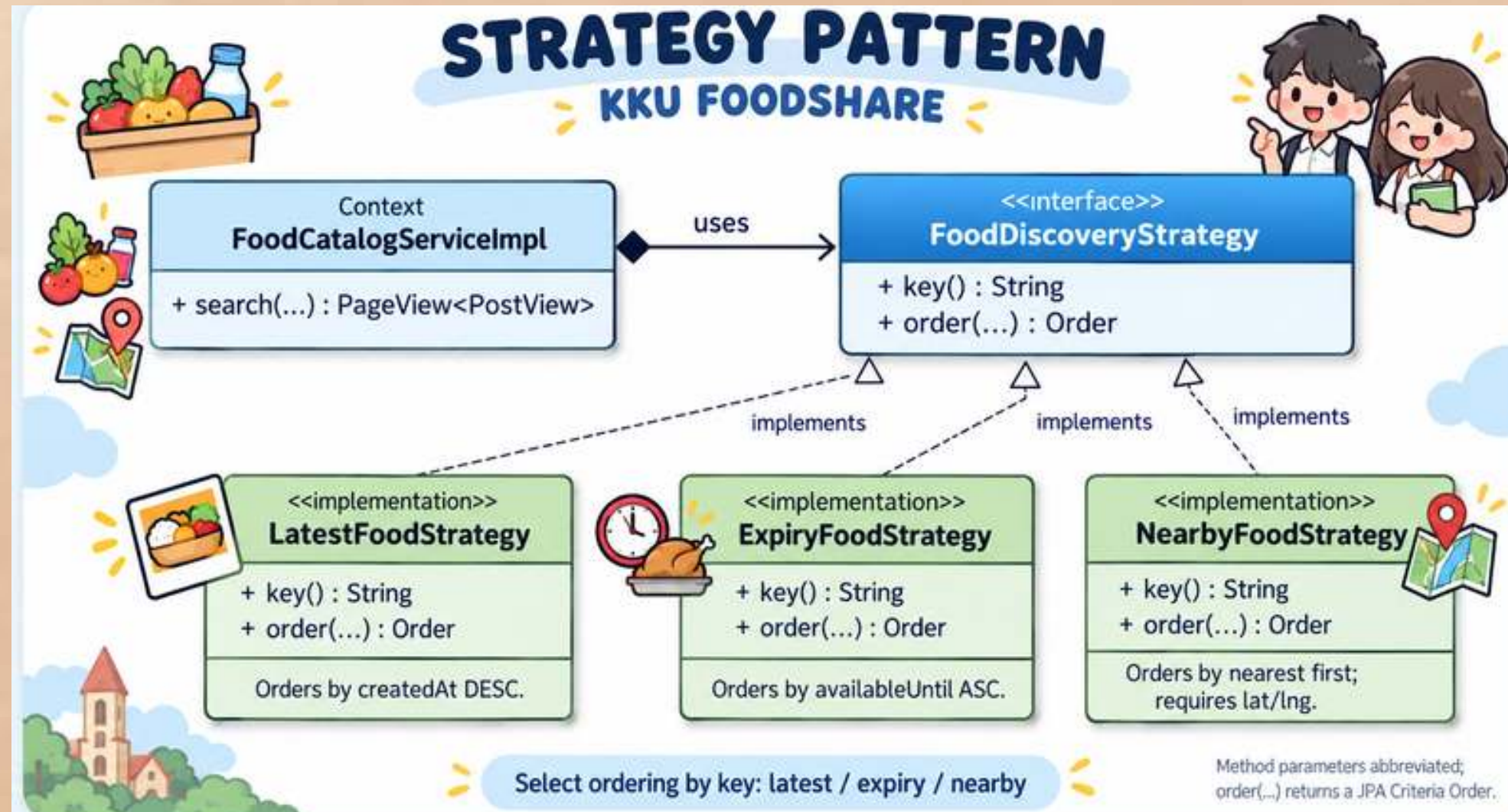
หลักการ	การประยุกต์ใช้งานในซอร์สโค้ด	ตำแหน่งไฟล์อ้างอิง
S (Single Responsibility)	แยก Controller, Service, Validation และ DTO ออกจากกันชัดเจน	service/FoodPostService.java
O (Open/Closed)	ใช้ Strategy Pattern สำหรับการค้นหา กรอง และเรียงลำดับ	strategy/FoodFilterStrategy.java
L (Liskov Substitution)	Subclass/Implementation ทำงานแทน Interface ได้สมบูรณ์ โดยไม่พังต่อ	service/impl/NotificationServiceImpl.java
I (Interface Segregation)	แยก Interface ย่อยเฉพาะทาง ไม่มี Fat Interface ที่มีเมธอดเกินความจำเป็น	repository/PickupRepository.java
D (Dependency Inversion)	High-level module พึ่งพา Interface + ใช้ Constructor Injection	controller/api/FoodPostApiController.java

Enterprise & Architectural Patterns

- Layered Architecture: แยก HTTP, Business Rules และ Data Access
- MVC: แยก UI หน้าเว็บจาก Controller และ Model
- Repository Pattern: ติดต่อฐานข้อมูลผ่าน Contract โดยไม่เขียน SQL ใน Controller
- Service Layer: รวม Authorization, Transaction และ Business Operations
- DTO + Mapper: ส่งเฉพาะข้อมูลตาม Contract ไม่ส่ง Entity ทั้งก้อนออกไป
- Dependency Injection: ใช้ Constructor Injection เพื่อรองรับการทำ Test

GOF DESIGN PATTERNS

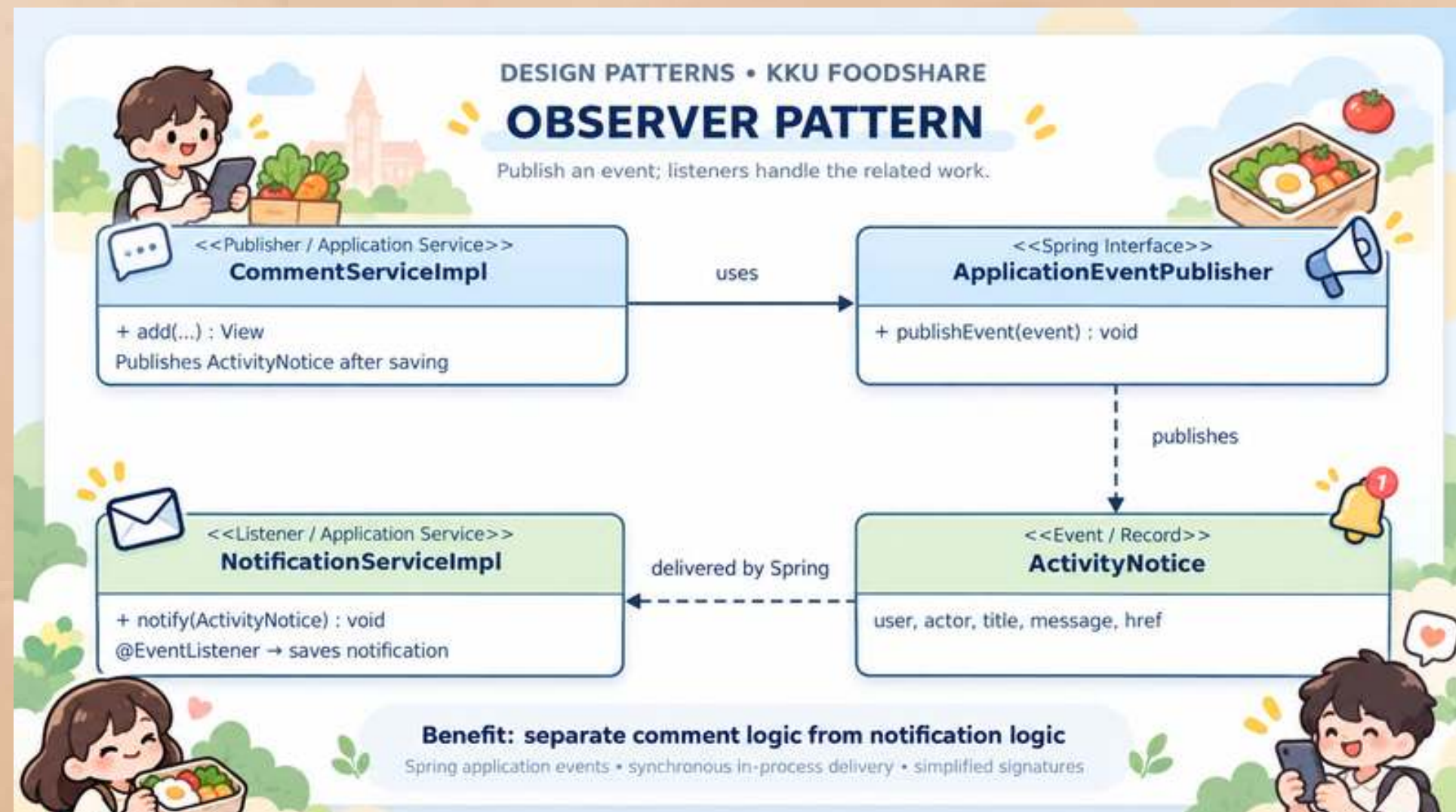
ระบบเลือกใช้ PATTERN ในกลุ่ม BEHAVIORAL



Strategy Pattern: แยกวิธีเรียงรายการอาหาร เช่น โพสต์ล่าสุด ใกล้เคียง หมดเวลารับ และใกล้เคียงผู้ใช้ ผ่าน Interface `FoodDiscoveryStrategy` โดย `FoodCatalogServiceImpl` เลือก implementation ตามตัวเลือกของผู้ใช้ ช่วยให้เพิ่มวิธีเรียงใหม่ได้โดยไม่ต้องเพิ่ม if-else ในส่วนเลือกวิธีเรียง

GOF DESIGN PATTERNS

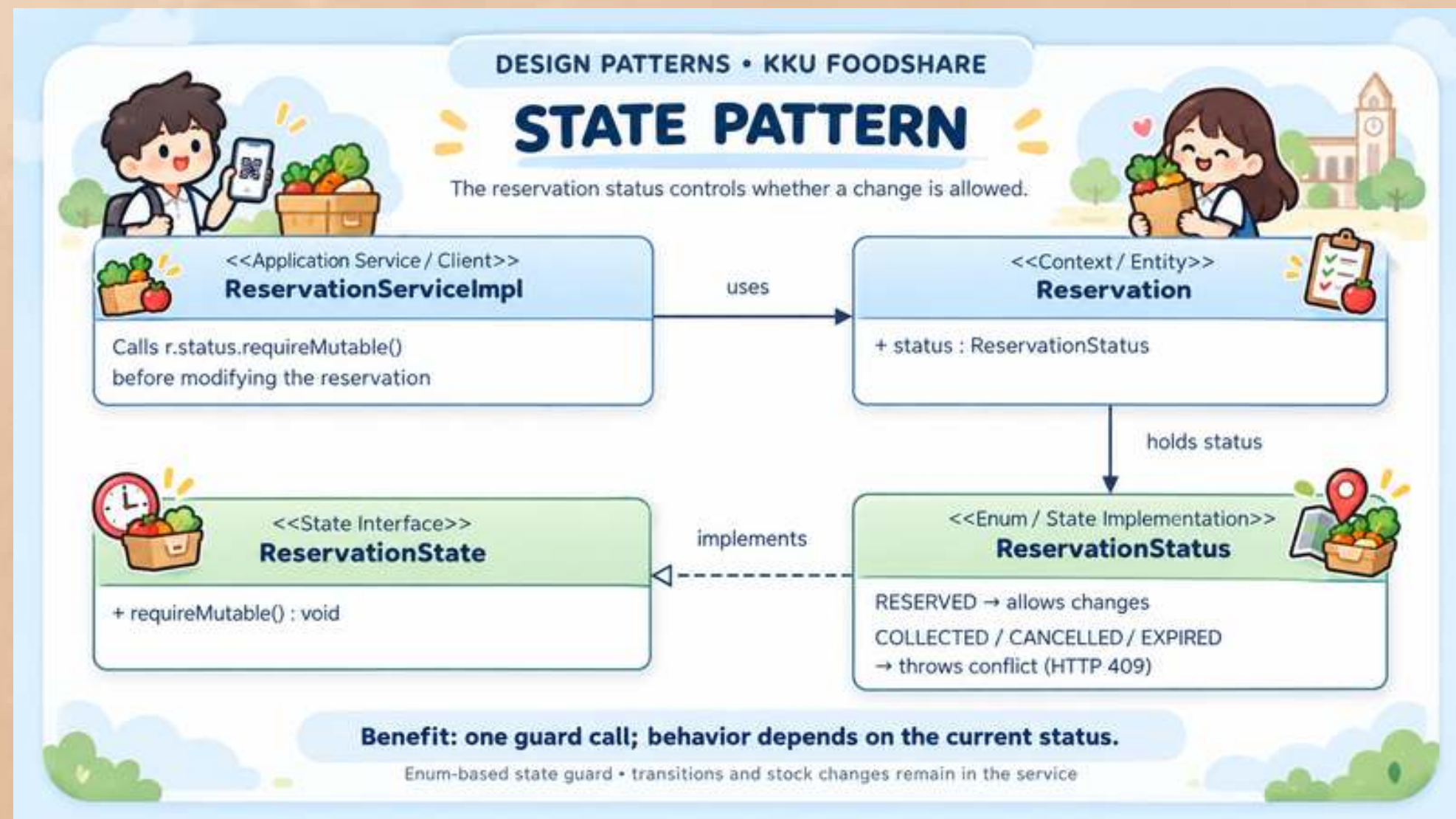
ระบบเลือกใช้ PATTERN ในกลุ่ม BEHAVIORAL



Observer Pattern: แยกงานที่เกิดตามหลังการกระทำออกจากงานหลัก เช่น เมื่อเพิ่มความคิดเห็น ระบบส่งเหตุการณ์ ActivityNotice ให้ NotificationServiceImpl รับผ่าน @EventListener เพื่อสร้างการแจ้งเตือน ช่วยลดการผูกกันระหว่างบริการ และรองรับการเพิ่มผู้ฟังเหตุการณ์อื่นในอนาคต

GOF DESIGN PATTERNS

ระบบเลือกใช้ PATTERN ในกลุ่ม BEHAVIORAL



State Pattern: กำหนดพฤติกรรมตามสถานะการจองผ่าน ReservationStatus ซึ่ง implement ReservationState โดย RESERVED อนุญาตให้เปลี่ยนข้อมูล ส่วน COLLECTED, CANCELLED และ EXPIRED ปฏิเสธผ่าน requireMutable() ช่วยลด การตรวจสอบสถานะซ้ำใน Service ทั้งนี้ การเปลี่ยนสถานะและปรับสต็อก ยังอยู่ใน Service

การออกแบบ ฐานข้อมูล

ข้อมูลหลักของระบบ



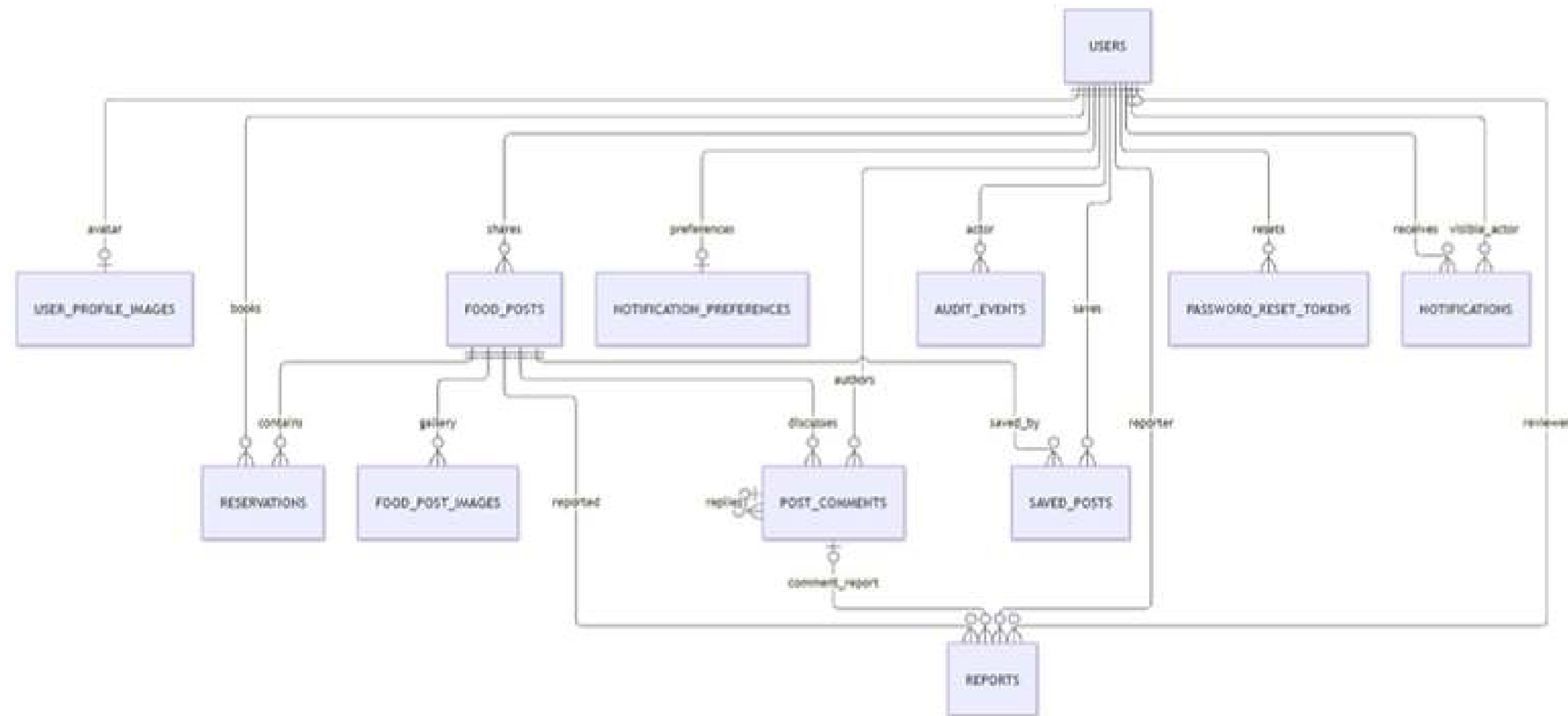
User
จัดเก็บข้อมูลผู้ใช้งาน



Food Post
จัดเก็บรายละเอียดอาหารที่นำมาแบ่งปัน



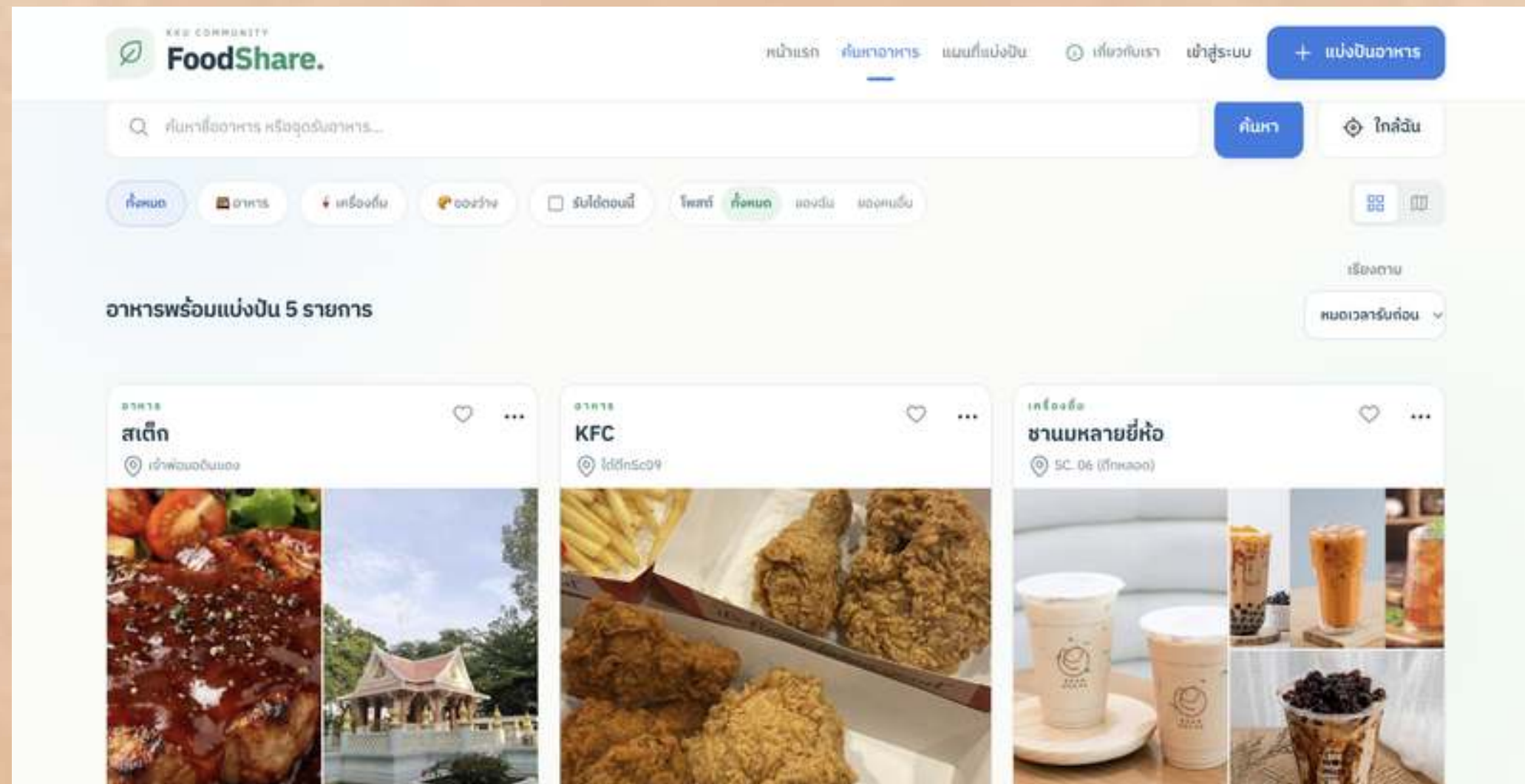
Pickup / Reservation
จัดเก็บข้อมูลการจองหรือรับอาหาร ตามโครงสร้างจริงของระบบ



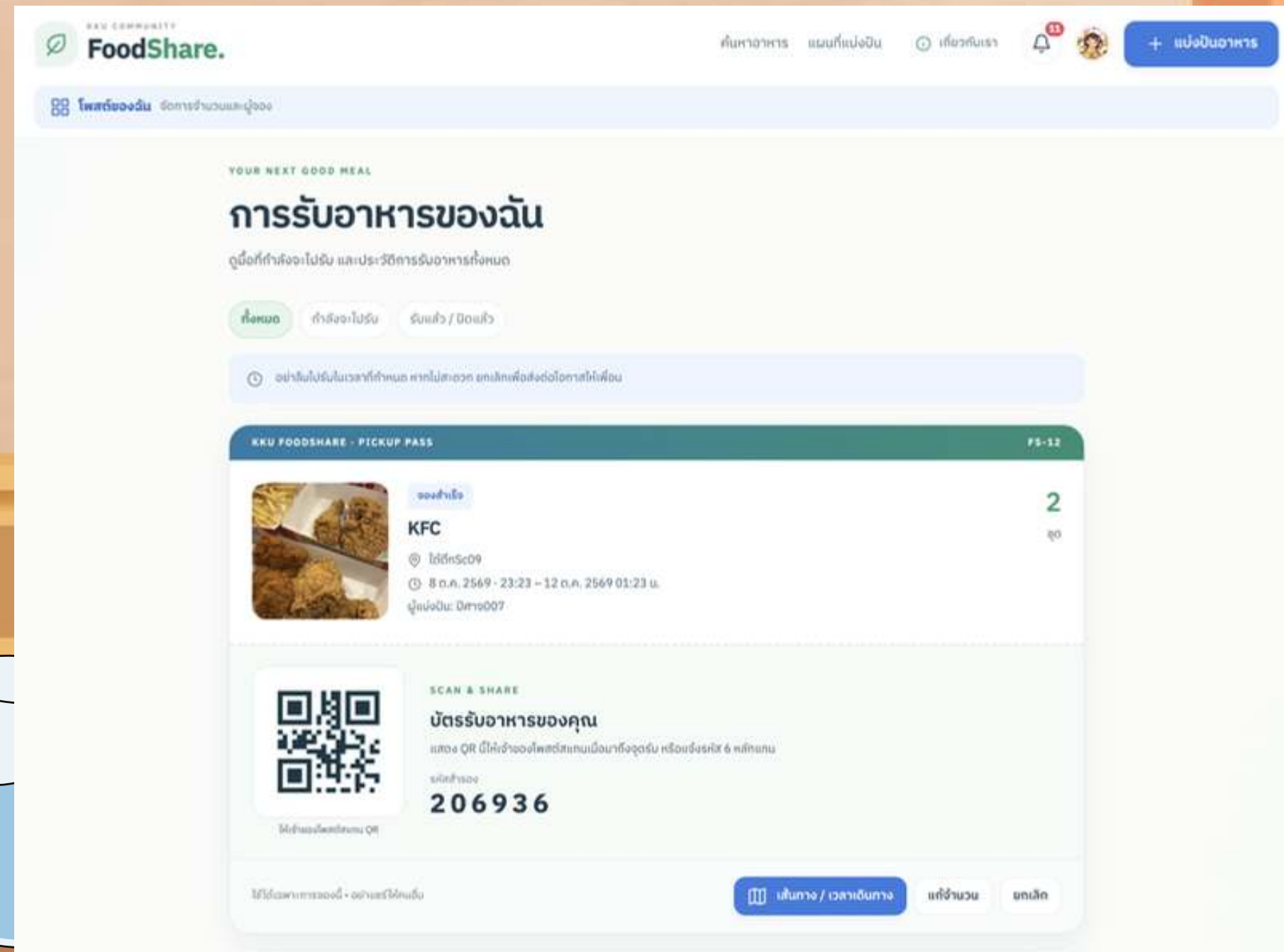
DEMO



ตัวอย่าง หน้าจอระบบ



การจองและ รับอาหาร



สรุปผล

ผลการพัฒนา



พัฒนาเว็บแอปพลิเคชันสำหรับโพสต์ ค้นหา จอง และยืนยันการรับอาหาร

ประโยชน์ที่ได้รับ



ช่วยให้สมาชิกแบ่งปันอาหารส่วนเกินและเข้าถึงอาหารที่ยังรับประทานได้สะดวกขึ้น

แนวทางพัฒนาต่อ



ปรับปรุงประสบการณ์ผู้ใช้ เพิ่มความน่าเชื่อถือของข้อมูล และพัฒนาระบบให้รองรับการใช้งานได้ดียิ่งขึ้น

ข้อจำกัด



1. ขอบเขตการใช้งาน
ออกแบบมาเพื่อสนับสนุนการแบ่งปันอาหารภายในมหาวิทยาลัยขอนแก่นเป็นหลัก



2. ความถูกต้องของข้อมูล
ข้อมูลอาหาร จำนวน เวลา และสถานที่รับ ขึ้นอยู่กับข้อมูลที่ผู้ใช้ระบุ



3. ความปลอดภัยของอาหาร
ระบบไม่สามารถตรวจสอบคุณภาพหรือความปลอดภัยของอาหารได้โดยตรง



4. การพึ่งพาบริการภายนอก
ฟังก์ชันบางอย่างขึ้นอยู่กับอินเทอร์เน็ตและความพร้อมของบริการที่เชื่อมต่อ



5. ข้อจำกัดด้านแผนที่และการค้นหาสถานที่
ระบบใช้บริการค้นหาสถานที่ทางเลือกแทน Google Maps API โดยเน้นพื้นที่มหาวิทยาลัยขอนแก่น จึงอาจค้นหาบางสถานที่ที่ไม่พบ ผู้ใช้สามารถปักหมุดเองและเปิด Google Maps เพื่อนำทางได้



THANK YOU

for listening